

Lecture 18 - Logistic Regression

Logistic Regression, also called *Logit* Regression or Logistic Discrimination

Note that, despite of its name, logistic regression is a model for classification, not regression.

The **logistic discriminant** is a linear model for binary classification that can be extended to multi-class classification using the one-vs-all approach.

Logistic regression is a **probabilistic discriminative classifier**.

In logistic discrimination, we do not model the class-conditional densities, $p(x|C_i)$, but rather their ratio. Let us assume we are working with a two-class problem and assume that the log likelihood ratio (or **odds ratio**) is linear:

$$\log \frac{P(x|C_1)}{P(x|C_2)} = \mathbf{w}^T x + b$$

- This is only true, if the classes are Gaussian-distributed!
- But logistic discrimination has a wider scope of applicability; for example, x may be composed of discrete attributes or may be a mixture of continuous and discrete attributes. Using Bayes' rule, we have:

$$\begin{aligned} \text{logit } P(C_1|x) &= \log \frac{P(C_1|x)}{1 - P(C_1|x)} \\ &= \log \frac{P(x|C_1)}{P(x|C_2)} + \log \frac{P(C_1)}{P(C_2)} \\ &= \mathbf{w}^T x + w_0 \end{aligned}$$

where $w_0 = b + \log \frac{P(C_1)}{P(C_2)}$.

Rearranging terms, we get the probabilistic classification:

$$y = P(C_1|x) = \frac{1}{1 + \exp(-(\mathbf{w}^T x + w_0))}$$

This is the sigmoid function:

$$\phi(z) = \frac{1}{1 + \exp(-z)}$$

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
plt.style.use('bmh')
```

```
In [ ]: z = np.linspace(-8,8,100)

plt.figure(figsize=(7,5))
plt.plot(z, 1/(1+np.exp(-z)))
plt.plot(z, [0.5]*len(z), '--')
plt.xlabel('$z$', size=20); plt.ylabel('$\phi(z)$', size=20);
plt.title('Sigmoid Function', size=15);
```

- We can see that $\phi(z) \rightarrow 1$ as $z \rightarrow \infty$, since $\exp(-z)$ becomes very small for large values of z .
- Similarly, $\phi(z) \rightarrow 0$ as $z \rightarrow -\infty$ as the result of an increasingly large denominator.

Thus, we conclude that this sigmoid function takes real number values as input and transforms them to values in the range $[0, 1]$ with an intercept at $\phi(z) = 0.5$.

```
In [1]: from IPython.display import Image
Image('figures/Figure4.4.png', width=700)
# Source: Bishop textbook
```

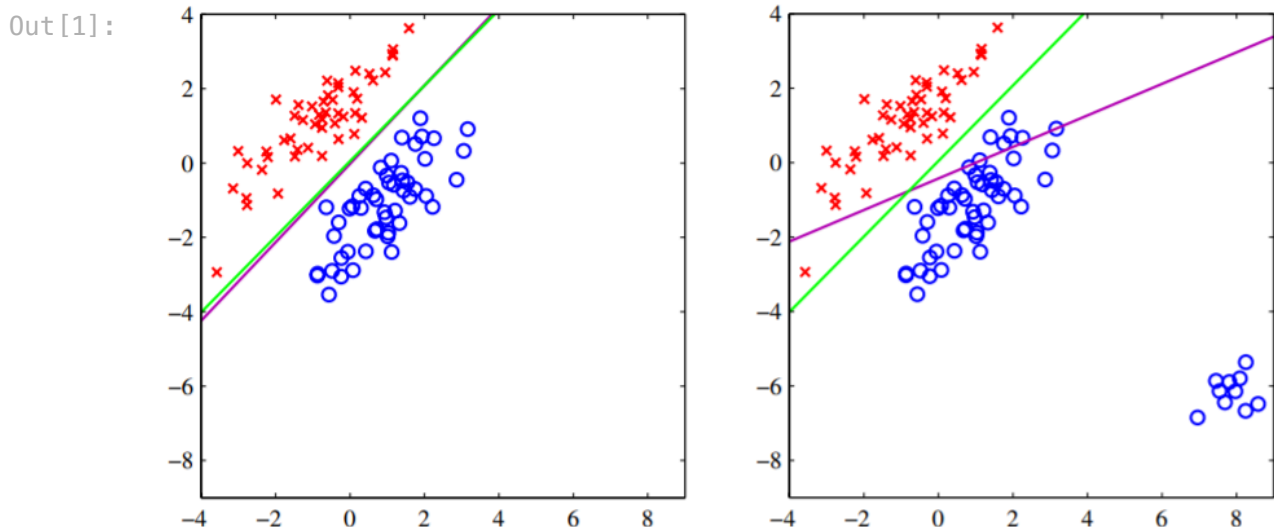


Figure 4.4 The left plot shows data from two classes, denoted by red crosses and blue circles, together with the decision boundary found by least squares (magenta curve) and also by the logistic regression model (green curve), which is discussed later in Section 4.3.2. The right-hand plot shows the corresponding results obtained when extra data points are added at the bottom left of the diagram, showing that least squares is highly sensitive to outliers, unlike logistic regression.

This is the same as using a sigmoid function as the **activation function** in the perceptron diagram.

Thus the output of the sigmoid function is then interpreted as the probability of particular sample belonging to C_1 , given its features x parameterized by the weights w .

- For example, if we compute $\phi(z) = 0.8$ for a particular sample, it means that the chance that this sample is in C_1 is 80%.

The predicted probability can then simply be converted into a binary outcome via a quantizer (unit step function):

$$\begin{aligned}\hat{t} = y &= \begin{cases} 1, & \phi(z) \geq 0.5 \\ 0, & \text{otherwise} \end{cases} \\ &= \begin{cases} 1, & \phi(\mathbf{w}^T x + w_0) \geq 0.5 \\ 0, & \text{otherwise} \end{cases}\end{aligned}$$

or,

$$\begin{aligned}\hat{t} = y &= \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \\ &= \begin{cases} 1, & \mathbf{w}^T x + w_0 \geq 0 \\ 0, & \mathbf{w}^T x + w_0 < 0 \end{cases}\end{aligned}$$

The Objective Function

Let $\{(x_i, t_i)\}_{i=1}^N$ be the set of input samples and its class labels, where $t_i \in \{0, 1\}$. Assuming the data samples are i.i.d., we can build the observed data likelihood:

$$\mathcal{L}^0 = \prod_{i=1}^N P(y_i | x_i; \mathbf{w}) = \phi(z_i)^{t_i} (1 - \phi(z_i))^{1-t_i}$$

We can apply the "trick" (log-likelihood) to build the data likelihood":

$$\mathcal{L} = \sum_{i=1}^N t_i \log \phi(z_i) + (1 - t_i) \log(1 - \phi(z_i))$$

where $\phi(z) = \frac{1}{1 + \exp(-z)}$ and $z_i = \mathbf{w}^T x_i + w_0$.

We want to maximize this likelihood to the data, or we can also write it as a minimization optimization:

$$J(\mathbf{w}, w_0) = \sum_{i=1}^N -t_i \log \phi(z_i) - (1 - t_i) \log(1 - \phi(z_i))$$

This objective function is also known as **cross-entropy**.

```
In [ ]: z = np.linspace(-2,2,100)

phi = lambda z: 1/(1+np.exp(-z))

plt.figure(figsize=(7,5))
plt.plot(phi(z), -np.log(phi(z)),label='when $t=1$')
plt.plot(phi(z), -np.log(1-phi(z)), '--',label='when $t=0$')
plt.legend(fontsize=15); plt.xlabel('$\phi(z)$',size=20)
plt.ylabel('$J(\mathbf{w})$',size=20);

In [ ]: plt.plot(z, -np.log(phi(z))-np.log(1-phi(z)), '--',label='$J(\mathbf{w},\mathbf{w}_0)$')
plt.legend(fontsize=20);
```

We can see that the cost approaches 0 if we correctly predict that a sample belongs to class 1. Similarly, we can see on the y axis that the cost also approaches 0 if we correctly predict class 0. However, if the prediction is wrong, the cost goes towards infinity: we penalize wrong predictions with an increasingly larger cost.

As we do not have the global *picture* of what the objective function, $J(\mathbf{w})$, we apply a search method to navigate through the objective function to find the *local optima* starting from an initial value, namely, **gradient descent**.

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}^{(t)}}$$

$$\mathbf{w}_0^{(t+1)} \leftarrow \mathbf{w}_0^{(t)} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_0^{(t)}}$$

where η is the learning rate (or step size).

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}^{(t)}} = - \sum_{i=1}^N \left[t_i \frac{1}{\phi(z_i)} \frac{\partial \phi(z_i)}{\partial z_i} \frac{\partial z_i}{\partial \mathbf{w}} - (1 - t_i) \frac{1}{1 - \phi(z_i)} \frac{\partial \phi(z_i)}{\partial z_i} \frac{\partial z_i}{\partial \mathbf{w}} \right]$$

where $\frac{\partial \phi(z_i)}{\partial z_i} = \phi'(z_i)$ and $\frac{\partial z_i}{\partial \mathbf{w}} = x_i$. Substituting:

$$\begin{aligned} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}^{(t)}} &= \sum_{i=1}^N -t_i \frac{\phi'(z_i)}{\phi(z_i)} x_i + (1 - t_i) \frac{\phi'(z_i)}{1 - \phi(z_i)} x_i \\ &= \sum_{i=1}^N \left(-\frac{t_i}{\phi(z_i)} + \frac{1 - t_i}{1 - \phi(z_i)} \right) \phi'(z_i) x_i \end{aligned}$$

where $\phi'(z_i) = \phi(z_i)(1 - \phi(z_i))$, then applying some substitutions we have:

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}^{(t)}} = - \sum_{i=1}^N (t_i - y_i) x_i$$

and, similarly,

$$\frac{\partial J(\mathbf{w})}{\partial w_0^{(t)}} = - \sum_{i=1}^N (t_i - y_i)$$

Finally,

$$\mathbf{w}^{(t+1)} \longleftarrow \mathbf{w}^{(t)} + \eta \sum_{i=1}^N (t_i - y_i) \mathbf{x}_i$$

$$\mathbf{w}_0^{(t+1)} \longleftarrow \mathbf{w}_0^{(t)} + \eta \sum_{i=1}^N (t_i - y_i)$$
